M

Databases

Authors

Matt ter Steege
Matttersteege@gmail.com

April 7, 2026

Contents

1 Relationele model & algebra **3**
1.1 defenities 3
1.2 Keys 3
1.3 Selectie & Projectie 3
1.4 Query optimalisatie 4
1.5 Query processing 5
2 Enity Relationship diagrams (ERD) **5**
3 Functional dependencies & normalisatie **5**
3.1 Functional dependencies 5
3.2 Decompositie 6
4 Normalisatie **6**
4.1 Indexen 6
4.2 Dependancy preserving (DP) 6
4.3 3NF 7
4.4 Boyce-codd normal form (BCNF) 7
4.5 4NF 7
4.6 Minimal cover calculation 8
5 Recovery & Logging **9**
5.1 Logging-types 9
5.2 Non-quiescent checkpoints (UNDO/REDO) 9
5.3 Herstel-algoritme 9
5.4 Voorbeeld 9
6 2-Phase Commit Protocol (2PC) **10**
6.1 Fase 1 — Stemmen 10
6.2 Fase 2 — Beslissing 10
6.3 Herstel na timeout 10
6.4 Cooperative Termination Protocol (CTP) 10
7 Concurrency **10**
7.1 Conflicts 10
7.2 Complete Precedence Graph 11
8 SQL **12**
8.1 DDL 12
8.2 DML (triggers) 14

8.3 DCL 14
8.4 SQL Injection 15
9 External merge sort en MapReduce 16
9.1 External merge sort 16
9.2 Kernidee 16
9.3 Stappenplan 16
9.4 MapReduce 16

1 | Relationele model & algebra

defenities

Relatieschema: Movie			
movid	**title**	**year**	**rating**
tt0469494	There will be blood	2007	8,1
tt0086879	Amadeus	1984	8,4
tt0102926	The silence of the lambs	1991	8,6
...	...	...	...

Uitleg gekoppeld aan de tabel:
- **Movie** is het **relatieschema** R $R(\text{movid}, \text{title}, \text{year}, \text{rating})$
- De **kolomnamen** zijn de **attributen** $attr(R) = \{\text{movid}, \text{title}, \text{year}, \text{rating}\}$
- Het aantal attributen is 4, dus de **degree** is 4
- Elke rij in de tabel is één **tuple** Bijvoorbeeld: $< \text{tt0469494}, \text{There will be blood}, 2007, 8{,}1 >$
- Elk attribuut heeft zijn **domein**:
  - movid $\rightarrow$ string
  - title $\rightarrow$ string
  - year $\rightarrow$ integer
  - rating $\rightarrow$ float
- Alle rijen samen vormen de **relatie** $r(Movie)$

Keys

- **Key** is een attribuut of combinatie van attributen die een tuple uniek identificeert binnen een relatie. Bijvoorbeeld: movid is een key voor Movie.
- **Superkey** is een attribuut of combinatie van attributen die een tuple uniek identificeert, maar kan ook extra attributen bevatten. Bijvoorbeeld: (movid, title) is een superkey voor Movie.
  Definitie: Een set van attributen $K \subseteq attr(R)$ is een superkey als voor elke relatie $r(R)$: $\forall t_1, t_2 \in r(R)$: $t_1[K] = t_2[K] \Rightarrow t_1 = t_2$. of: "Als twee tuples dezelfde waarden hebben voor alle attributen in K, dan zijn het dezelfde tuples."
- **Candidate key** is een minimale superkey, het is de minimale set (kan ook één attribuut zijn) die alle andere attributen identificeert. Dit mag direct en indirect. $A \rightarrow B, B \rightarrow C$ betekend dat A een candidate key is, omdat $A^+ = ABC$ en dus elk attribuut omvat.
- **Prime keys** (niet primary key) is elke key die binnen een candidate key vallen.
- **Primary key** is de gekozen candidate key die wordt gebruikt om tuples uniek te identificeren binnen een relatie. Bijvoorbeeld: movid is de primary key voor Movie.
- **Foreign key** is een attribuut of combinatie van attributen in een relatie die verwijst naar de primary key van een andere relatie. Bijvoorbeeld: in een relatie Actor met attribuut movid als foreign key die verwijst naar de primary key movid in Movie. Dit kan je gebruiken om aan te geven in welke films een acteur heeft gespeeld.
  Definitie: Een set attributen $K \in attr(R)$ is een foreign key van relatie R_j als candidate key. $R_i[K]$ verwijst naar de primary key van relatie R_j. of: "De waarden van K in R_i moeten overeenkomen met de waarden van de primary key in R_j."
- **Key constraints** zorgen ervoor dat de waarden van de key-attributen uniek zijn binnen de relatie en niet null mogen zijn.
- De **referential integrity constraints** zorgen ervoor dat de waarden van de foreign key-attributen overeenkomen met de waarden van de primary key in de gerelateerde relatie, of null zijn als er geen overeenkomst is.

Selectie & Projectie

- **Selectie** (σ_p), σ is een operator die rijen (tuples) selecteert uit een relatie op basis van het predicaat p.
- **and** ($\wedge$) en **or** ($\vee$) worden gebruikt om meerdere voorwaarden te combineren in het predicaat. Voorbeeld: $\sigma_{\text{year}>2000 \wedge \text{rating}>8}(Movie)$ selecteert alle films uit de Movie-relatie die zijn uitgebracht na het jaar 2000 en een beoordeling hoger dan 8 hebben.

- **Projectie** (π_L), π is een operator die kolommen (attributen) selecteert uit een relatie op basis van de lijst van attributen L. Voorbeeld: $\pi_{\text{title,year}}(Movie)$ projecteert alleen de titel en het jaar van de films uit de Movie-relatie.
- **union** ($\cup$), de unie-operator combineert de resultaten van twee relaties door alle unieke tuples uit beide relaties te behouden. Voorbeeld: $R_1 \cup R_2$ geeft alle unieke tuples die voorkomen in zowel relatie R_1 als relatie R_2.
- **intersection** ($\cap$), de intersectie-operator retourneert de tuples die voorkomen in beide relaties. Voorbeeld: $R_1 \cap R_2$ geeft alle tuples die in zowel relatie R_1 als relatie R_2 voorkomen.
- **set difference** ($-$), de verschil-operator retourneert de tuples die voorkomen in de eerste relatie maar niet in de tweede. Voorbeeld: $R_1 - R_2$ geeft alle tuples die in relatie R_1 voorkomen maar niet in relatie R_2.
- **cartesian product** ($\times$), het cartesisch product combineert elke tuple van de eerste relatie met elke tuple van de tweede relatie. Voorbeeld: $R_1 \times R_2$ geeft alle mogelijke combinaties van tuples uit relatie R_1 en relatie R_2.
- **thetha-join** ($\bowtie_\theta$), de theta-join-operator combineert tuples uit twee relaties op basis van een opgegeven voorwaarde θ. Voorbeeld: $R_1 \bowtie_{\text{R1.attr=R2.attr}} R_2$ combineert tuples uit relatie R_1 en relatie R_2 waar de waarden van het attribuut attr gelijk zijn.
- **natural join** ($\bowtie$), de natural join-operator combineert tuples uit twee relaties op basis van gemeenschappelijke attributen met dezelfde naam. Voorbeeld: $R_1 \bowtie R_2$ combineert tuples uit relatie R_1 en relatie R_2 waar de waarden van de gemeenschappelijke attributen gelijk zijn.
- **division** ($\div$), de division-operator retourneert de tuples uit de eerste relatie die overeenkomen met alle tuples in de tweede relatie. Voorbeeld: $R_1 \div R_2$ geeft alle tuples uit relatie R_1 die overeenkomen met alle tuples in relatie R_2 op basis van een gemeenschappelijk attribuut.
- **Assignment & renaming** $T :=$<alg_expr>, de assignment-operator wijst het resultaat van een relationele algebra-expressie toe aan een nieuwe relatie T. Voorbeeld: $T := \sigma_{\text{year}>2000}(Movie)$ creëert een nieuwe relatie T die alle films bevat die zijn uitgebracht na het jaar 2000.
- **Renaming on the fly** ($\rho(T)$(<alg_expr>), de renaming-operator wijst het resultaat van een relationele algebra-expressie toe aan een nieuwe relatie T zonder een permanente naam te creëren. Voorbeeld: $\rho(T)(\sigma_{\text{year}>2000}(Movie))$ creëert een tijdelijke relatie T die alle films bevat die zijn uitgebracht na het jaar 2000, maar deze relatie is alleen beschikbaar binnen de context van deze expressie.

Voorbeeld **(bibliotheek schema)**:

Stel je hebt de volgende schema's voor een bibliotheekdatabase:
- **Book**(book_id, title, author, year)
- **Member**(member_id, name, join_date)
- **Loan**(loan_id, book_id, member_id, loan_date, return_date)

Geef alle namen van lezers die minstens één boek hebben geleend van Dickens.

$$\pi_{\text{name}}(\text{Reader} \bowtie (\text{Loan} \bowtie \sigma_{\text{author}='Dickens'}(\text{Book})))$$

Uitleg:
- $\sigma_{\text{author}='Dickens'}(\text{Book})$ selecteert alle boeken geschreven door Dickens.
- Loan $\bowtie \sigma_{\text{author}='Dickens'}(\text{Book})$ combineert de leningen met de geselecteerde boeken om alle leningen van Dickens' boeken te vinden.
- Reader $\bowtie$ (Loan $\bowtie \sigma_{\text{author}='Dickens'}(\text{Book})$) combineert de lezers met de leningen van Dickens' boeken om alle lezers te vinden die deze boeken hebben geleend.
- $\pi_{\text{name}}(\ldots)$ projecteert alleen de namen van deze lezers.

Query optimalisatie

XRA (eXtended Relational Algebra) is bedoeld om met multisets, sorteren en groeperen. Deze notatie kan je gebruiken in projections ($\pi A, A + B(R)$) om ze wat uitgebreider te maken.

Dit is een beetje zoals logica, je hebt Union ($\cup$), maar nu zijn dezelfde letters wel echt verschillende items. Bijvoorbeeld $\{a, b, c\} \cup \{a, a, c, d\} = \{a, a, a, b, c, c, d\}$ is een voorbeeld van union. Intersection ($\cap$): $\{a, a, a, b, c, c, d\} \cap \{a, a, b, e\} = \{a, a, b\}$. En als laatste de Minus ($-$), deze haalt het rechter deel uit het linkerdeel: $\{a, a, a, b, c, c, d\} - \{a, a, b, e\} = \{a, c, c, d\}$.

Deze (en andere) kan je dus in de projections zetten om je dat dus tijdens de projection aan te passen, ($\pi A, A + B(R)$) geeft je dus een resulaat dat A bevat en en tweede kolom waar A + B is gedaan.

τ wordt gebruik voor volgorden, als je zegt: $\tau_{\uparrow B \downarrow A}$ word in eerste instantie gesorteerd op kolom B van klein naar groot en daarna kolom A van groot naar klein, opvolgende sorteringen worden alleen binnen de groep waar B gelijk is aan elkaar gedaan.

Γ wordt gebruikt voor groeperen, zo kan je op kolom A groeperen en dan opvragen wat de minimale waarden van B is voor elke verschillende A: $\Gamma_{A,MIN(B)(R)}$. Er wordt dus gekeken naar elke groep waar A gelijk is aan elkaar, daarna wordt gekeken welke waarde B is het laagste per verschillende A waarde.

Het is altijd belangrijk om unary operators eerder te doen dan dit soort "zware" operators, want dan is de hoeveelheid data waarover de zware operators heen moeten gaan minder en gaat een operatie dus sneller.

Query processing

2 | Enity Relationship diagrams (ERD)

Als je een database wilt maken, dan is het belangrijk dat je het domein goed modelleert. Een ERD is een grafische weergave van de entiteiten, attributen en relaties binnen een domein. Het helpt bij het ontwerpen van databases door de structuur en de onderlinge verbanden duidelijk te maken.

ER diagrammen hebben verschillende componenten:
- **Rechthoek** Geeft een entiteit weer, bijvoorbeeld een persoon, plaats of ding.
- **Ovaal** Geeft een attribuut weer, bijvoorbeeld een naam, leeftijd of adres.
- **Ruit** Geeft een relatie weer tussen entiteiten, bijvoorbeeld "werkt bij" of "heeft".
- **Lijn** Verbindt entiteiten, attributen en relaties om de structuur van het domein te tonen.

Deze lijnen/relaties kunnen ook kardinaliteiten hebben, zoals 1:1, 1:N of M:N, om aan te geven hoeveel entiteiten er betrokken zijn bij de relatie.
1:1 (één-op-één) betekent dat één entiteit van het ene type overeenkomt met precies één entiteit van het andere type.
1:N (één-op-veel) betekent dat één entiteit van het ene type overeenkomt met meerdere entiteiten van het andere type.
M:N (veel-op-veel) betekent dat meerdere entiteiten van het ene type overeenkomen met meerdere entiteiten van het andere type.

Ook kan je deelname aan een relatie aangeven als verplicht (1,...) of optioneel (0,...), wat aangeeft of een entiteit deel moet nemen aan de relatie of niet.

Zwakke entiteiten kunnen alleen bestaan in relatie tot een andere entiteit, en hebben een gedeeltelijke sleutel die samen met de sleutel van de gerelateerde entiteit een unieke identificatie vormt. Bijvoorbeeld een afboeking als je iets koopt kan alleen als je een bankrekening hebt, en de afboeking heeft een gedeeltelijke sleutel die samen met de rekeningnummer van de bankrekening een unieke identificatie vormt.

3 | Functional dependencies & normalisatie

Functional dependencies

Stel je hebt een universum $R(A, B, C, D)$. Je hebt de regels $A \to B, B \to C, A \to C$

Dan kan je concluderen dat $A \to AB \to ABC$, ofwel $A \to ABC$.

Stel je hebt $A \to BCD$, dan kan je zeggen $A \to B$

Voorbeeld **(functionele afhankelijkheden)**:

We hebben een schema $R(ABCDEFG)$ en een set FD's: $F = \{C \to DE, A \to C, G \to D, B \to G\}$
We willen de closure van A berekenen, oftewel A^+, om te zien welke attributen functioneel afhankelijk zijn van A.
We beginnen met $A^+ = \{A\}$ (omdat A altijd functioneel afhankelijk is van zichzelf).
We kijken naar de FD's in F en zien dat $A \to C$ in F zit, dus we voegen C toe aan A^+: $A^+ = \{A, C\}$.
Nu kijken we opnieuw naar de FD's en zien dat $C \to DE$ in F zit, en aangezien C al in A^+ zit, kunnen we D en E ook toevoegen aan A^+: $A^+ = \{A, C, D, E\}$.
We kijken nogmaals naar de FD's, maar er zijn geen FD's meer die we kunnen toepassen op de attributen in A^+, dus we zijn klaar.
Stel wel willen $(AB)^+$ weten (notitie, dat is dus NIET AB^+, want dat is $A(B)^+$)

Decompositie

Voorbeeld **(XYZ-regel (Verliesvrij))**:

Gegeven $R(ABCDE)$ en FD's $F = \{A \to C,\ C \to BE,\ A \to E\}$.
Beschouw de decompositie $R_1 = (AC)$ en $R_2 = (ABDE)$.

De gemeenschappelijke attributen zijn $X = R_1 \cap R_2 = \{A\}$.
Schrijf $Y = R_1 \setminus X = \{C\}$ en $Z = R_2 \setminus X = \{B, D, E\}$.

Volgens de XYZ-regel is de decompositie verliesvrij als $X \to Y$ of $X \to Z$ geldt (met betrekking tot F^+).
Controleer $A \to C$ en $A \to BDE$:
uit F volgt $A \to C$ en $A \to E$, maar er is geen $A \to B$ en geen $A \to D$.
Dus geldt $A \to C$ maar niet $A \to BDE$.

Conclusie: de decompositie (AC) $(ABDE)$ is verliesvrij volgens de gegeven FD's.

Voorbeeld **(XYZ-regel (niet verliesvrij))**:

Gegeven $R(ABCDE)$ en FD's $F = \{A \to C,\ C \to BE,\ A \to E\}$.
Beschouw de decompositie $R_1 = (AC)$ en $R_2 = (CBDE)$.

De gemeenschappelijke attributen zijn $X = R_1 \cap R_2 = \{C\}$.
Schrijf $Y = R_1 \setminus X = \{A\}$ en $Z = R_2 \setminus X = \{B, D, E\}$.

Volgens de XYZ-regel is de decompositie verliesvrij als $X \to Y$ of $X \to Z$ geldt (met betrekking tot F^+).
Controleer $C \to A$ en $C \to BDE$:
uit F volgt $C \to B$ en $C \to E$, maar er is geen $C \to A$ en geen $C \to D$.
Dus geldt noch $C \to A$ noch $C \to BDE$.

Conclusie: de decompositie (AC) $(CBDE)$ is niet verliesvrij volgens de gegeven FD's.

4 | Normalisatie

Tijdens het normaliseren is het de bedoeling dat je de database opsplitst in meerdere tabellen om: **1.** redundantie te verminderen, **2.** lossless te zijn.

Indexen

Indexen zijn datastructuren die worden gebruikt om de snelheid van gegevensopvragingen in een database te verbeteren. Ze werken als een soort inhoudsopgave voor de database, waardoor het sneller is om specifieke records te vinden zonder de hele tabel te hoeven doorzoeken. Indexen kunnen worden gemaakt op één of meer kolommen van een tabel, en ze kunnen verschillende vormen aannemen, zoals B-tree indexen, hash indexen, of bitmap indexen, afhankelijk van het type gegevens en de query's die worden uitgevoerd. Het gebruik van indexen kan de prestaties van zoekopdrachten aanzienlijk verbeteren, vooral bij grote datasets, maar het kan ook extra opslagruimte vereisen en de prestaties van schrijfoperaties (zoals INSERT, UPDATE, DELETE) verminderen, omdat de indexen ook moeten worden bijgewerkt.

Dependancy preserving (DP)

Om te kijken of iets dependency preserving is moet je de originele set van FD's kunnen herleiden uit de decomposities.

Stel je het volgende voorbeeld voor, je hebt de relatie $R(ABC)$ met de FD's $F = \{A \to B, B \to C\}$. De decompositie is $R_1(AB)$ & $R_2(BC)$. Je moet nu gaan kijken welke functionele dependancies binnen één enkele relatie blijft, dus alle letters in een FD moeten in één relatie zitten. In dit geval blijft de relatie $A \to B$ in relatie R_1 en $B \to C$ in relatie R_2. Deze vormen samen dezelfde set als de orignele FD's, dus is deze decompositie

dependancy preserving.

Een ander voorbeeld is $R(ABCD)$ met FD's $F = \{AB \to C, C \to D, D \to A\}$. De decompositie hier is $R_1(ABC)$ en $R_2(CD)$. ALs je dezelfde stappen toepast krijg je dat R_1 de functional dependancy $AB \to C$ waarborgt en R_2 de FD $C \to D$ waarborgt. Als je deze samenvoegt krijg je dus $F' = \{AB \to C, C \to D\}$, maar uit deze set FD's kan je niet de orignele FD $D \to A$ halen en is deze decompositie dus **niet** dependancy preserving.

3NF

3NF moet zich houden aan de volgende regels, het moet voldoen aan 2NF, 1NF en belangrijker: er mag **geen transitieve dependency zijn voor niet-prime attributen**. Dit betekend dat alle attributen die in een relatie zitten afhankelijk moeten zijn van een primary key. Transitieve dependency is eigenlijk: $A \to B, B \to C$, $A \to C$ is dan een transitieve dependency, hij is niet direct. Dit mag dus absoluut niet in 3NF (en hoger). 3NF staat wel toe dat dat in de dependency $X \to Y$ (lees bijvoorbeeld ook $AB \to C$ of $A \to BC$) X een superkey is. Een andere optie is dat Y een prime attribute is, dit betekend dat elk attribuut dat onder Y valt (dus bijvoorbeeld het BC gedeelte in $A \to BC$) onderdeel is van een candidate key is.

Voorbeeld **(3NF)**:

Stel je hebt de relatie $R(ABCDE)$ en de FD's $F = \{A \to BCE, C \to D\}$. Hieruit kan je opmaken dat A de candidate key is, want $A^+ = ABCDE$. Echter is er wel een transitive dependency, namelijk $A \to D$, dat mag in 3NF niet. Om dit 3NF te maken, moet je de relatie dus 1 keer opsplitsen in $R_1(ABCE)$ en $R_2(CD)$.

Als 3NF niet houdt, dan houdt BCNF en hoger dus automatisch ook niet.

Boyce-codd normal form (BCNF)

BCNF moet zich houden aan de volgende regels, het moet voldoen aan 3NF en dat in de dependency $X \to Y$ (lees bijvoorbeeld ook $AB \to C$ of $A \to BC$) X een superkey is.

Als je snel wilt verifiëren of iets in BCNF is moet je kijken naar de linkerzijde van de FD's, zijn dit allemaal candidate keys, dan is het in BCNF.

4NF

4NF lijkt op BCNF, maar kijkt niet naar functionele dependencies ($X \to Y$), maar naar **multivalued dependencies** ($X \twoheadrightarrow Y$).

Een multivalued dependency betekent dat voor één waarde van X meerdere waarden van Y kunnen bestaan, die **onafhankelijk** zijn van andere attributen. Dit zorgt ervoor dat combinaties ontstaan die eigenlijk niet bedoeld zijn (alles wordt met alles gecombineerd).

4NF voorkomt dit door te eisen dat bij elke non-triviale multivalued dependency $X \twoheadrightarrow Y$ geldt dat X een superkey is. Is dat niet zo, dan moet je de relatie opsplitsen.

Voorbeeld **(4NF)**:

Stel $R(Student, Hobby, Taal)$ met: $Student \Rightarrow Hobby$ en $Student \Rightarrow Taal$.
Dit betekent dat hobby's en talen los van elkaar staan, maar in één tabel worden ze met elkaar gecombineerd.
Om dit op te lossen splits je: $R_1(Student, Hobby)$ en $R_2(Student, Taal)$.

Minimal cover calculation

Een minimal cover van een set functionele afhankelijkheden is een equivalentie van die set waarbij alle functionele afhankelijkheden in de minimal cover minimaal zijn, wat betekent dat er geen overbodige functionele afhankelijkheden zijn en dat de rechterkant van elke functionele afhankelijkheid slechts één attribuut bevat. Het berekenen van een minimal cover gaat als volgt:
Stel je hebt de volgende set functionele afhankelijkheden:

$$F = \{A \to D, B \to HA, GH \to AKL, CH \to BK, G \to BC\}$$

Calculate a minimal cover for **F**

Stap 1 is om de rechterkant van elke functionele afhankelijkheid te splitsen in enkelvoudige attributen:

- A → D
- B → H
- B → A
- GH → A
- GH → K
- GH → L
- CH → B
- CH → K
- G → B
- G → C

Stap 2 is de linkerkant van elke functionele afhankelijkheid minimaliseren door te controleren of er overbodige attributen zijn:

- A → D
- B → H
- B → A
- GH → A ($G^+ = GBCHAKLD$, dus G → A)
- GK → A ($G^+ = GBCHAKLD$, dus G → K)
- GL → A ($G^+ = GBCHAKLD$, dus G → L)
- CH → B (Kan niet versimpeld worden, want $C^+ \to \{C\}$ en $H^+ \to \{H\}$)
- CH → K (Kan niet versimpeld worden, want $C^+ \to \{C\}$ en $H^+ \to \{H\}$)
- G → B
- G → C

Stap 3 is om te kijken over er overbodige functionele afhankelijkheden zijn door te controleren of ze kunnen worden afgeleid uit de andere functionele afhankelijkheden:

- A → D
- B → H
- B → A
- G → A (kan worden afgeleid uit G → B en B → A, dus overbodig)
- G → K (kan worden afgeleid uit G → C en C → K, dus overbodig)
- G → L (kan *niet meer* worden afgeleid, want G → A is al weg)
- CH → B
- CH → K
- G → B
- G → C

De minimal cover van F is dus:

$$F_{min} = \{A \to D, B \to HA, G \to LBC, CH \to BK\}$$

(opmerking: G → LBC is een notatie die betekent dat G → L, G → B en G → C allemaal in de minimal cover zitten)

5 | Recovery & Logging

Logging-types

Bij een crash moet de database hersteld worden. Er zijn drie typen:

Type	Volgorde op schijf	UNDO bij crash?	REDO bij crash?
UNDO	log → data → COMMIT	Ja (uncommitted)	Nee
REDO	log → COMMIT → data	Nee	Ja (committed)
UNDO/REDO	log → data (COMMIT overal)	Ja (uncommitted)	Ja (committed)

Non-quiescent checkpoints (UNDO/REDO)

1. Schrijf `<START CKPT(T1,...,Tk)>` met de actieve transacties, flush de log.
2. Flush alle buffers die *vóór* START CKPT waren aangemaakt.
3. Schrijf `<END CKPT>`, flush de log.

Herstel-algoritme

Scan de log **achteruit** vanuit het laatste record:
- Eerste checkpoint = `END CKPT` ⇒ scan terug tot bijbehorende `START CKPT`.
- Eerste checkpoint = `START CKPT` (geen `END CKPT` gezien) ⇒ scan terug naar het **begin**.

Daarna: **REDO** alle committed transacties (vroegste eerst), dan **UNDO** alle uncommitted (laatste eerst).

Voorbeeld

```
<START T1>
<T1, A, 5, 15>
<START T2>
<T2, B, 10, 11>
<COMMIT T1>
<START CKPT (T2)>
<T2, C, 15, 16>
<START T3>
      ⋮
<T3, D, 20, 12>
<START T4>
<T4, E, 25, 26>
<COMMIT T2>
<END CKPT>
<T3, F, 30, 18>
<COMMIT T4>
<START T5>
<T5, G, 35, 38>
```

Crash na `<COMMIT T2>` — geen `END CKPT` gezien ⇒ scan terug tot begin:

Actie	Transacties	Reden
REDO	T1, T2	Gecommit vóór de crash
UNDO	T3, T4, T5	Niet gecommit (T4 en T5 gestart ná START CKPT, ook undoen)

Crash na `<T5, G, 35, 38>` — `END CKPT` gezien ⇒ scan terug tot `START CKPT(T2)`:

Actie	Transacties	Reden
REDO	T2, T4	Committed, maakten wijzigingen *ná* START CKPT
UNDO	T3, T5	Niet gecommit
Niet REDO	T1	Committed *vóór* START CKPT ⇒ al geflusht

6 | 2-Phase Commit Protocol (2PC)

Fase 1 — Stemmen

1. COORD schrijft `<prepare T>` in DT-log, stuurt `[prepare T]` naar alle PARTS.
2. Elke PART stemt: `[abort T]` (log + rollback) **of** `[ready T]` (log `<Ready T>`, stuur bericht).

Fase 2 — Beslissing

1. Alle stemmen positief: COORD logt `<Commit T>`, stuurt `[Commit T]` naar alle PARTS.
2. Anders: COORD logt `<Abort T>`, stuurt `[Abort T]` naar PARTS die `[Ready T]` stemden.
3. Elke PART die `[Ready T]` stemde wacht op beslissing, logt die en handelt ernaar.

Herstel na timeout

DT-log status	Rol	Actie
`<Commit T>` of `<Abort T>`	beide	Beslissing al genomen ⇒ gewoon uitvoeren.
`<Prepare T>`, geen beslissing	COORD	Nog geen beslissing ⇒ forceer Abort, hervat protocol.
Geen `<Ready T>`, geen beslissing	PART	Nog niet gestemd ⇒ forceer Abort.
`<Ready T>`, geen beslissing	PART	Vraag beslissing aan COORD. Geen reactie ⇒ start **CTP**.

Cooperative Termination Protocol (CTP)

Kies een leider onder de overgebleven PARTS. De leider vraagt status bij alle PARTS:
- Iemand kent de beslissing ⇒ broadcast die beslissing.
- Iemand heeft nog niet gestemd ⇒ broadcast Abort.
- Iedereen positief gestemd, niemand kent de beslissing ⇒ **GEBLOKKEERD** (menselijke interventie nodig).

2PC is een blocking protocol. 3PC vermindert blokkering, maar kan het niet volledig voorkomen bij netwerkstoringen.

7 | Concurrency

Concurrency betekent dat meerdere transacties gelijktijdig (interleaved) toegang hebben tot dezelfde data. Dit leidt tot problemen wanneer read- en write-operaties door elkaar heen lopen.

Een **schedule** is de chronologische volgorde van operaties (read/write) van transacties.

Er zijn twee types:
- **Serial schedule**: transacties worden volledig na elkaar uitgevoerd (altijd correct)
- **Interleaved schedule**: operaties van transacties worden gemengd

Doel: interleaved schedules moeten equivalent zijn aan een serial schedule (=**serialisable**).

Conflicts

Twee operaties zijn **conflicting** (een edge) als:
- ze op hetzelfde data-item werken, en
- minstens één een write is

Voorbeelden:
- $R(x)$ en $W(x)$ → conflict
- $W(x)$ en $W(x)$ → conflict
- $R(x)$ en $R(x)$ → geen conflict

Voorbeeld **(Schedule)**:

T1	T2	T3	T4	T5
$R_1(x)$				
	$R_2(x)$			
$W_1(x)$				
		$R_3(x)$		
	$W_2(x)$			
			$R_4(y)$	
				$R_5(y)$
		$W_3(x)$		
			$W_4(y)$	
$W_1(y)$				

Stap 1: zoek conflicten
Maak een chronologische lijst van alle operaties. Daarna ga je van boven naar beneden werken en stel je drie vragen, deze gaan over de huidige operatie en elke operatie ervoor apart:
1. Is het hetzelfde data-item $(X, Y, \cdots)$
2. Is het een andere transactie $(T1, T2, T3, \cdots)$
3. Is minstens één van de twee een write operatie $(W(\cdots)$

Alleen als alle drie de vragen "ja" zijn heb je een conflict en voeg je een pijl toe van de **oudere** naar de **nieuwe** operatie toe.
Conflicten op x:
- $R_1(x)$ en $W_2(x) \rightarrow$ conflict $\Rightarrow T_1 \rightarrow T_2$
- $R_1(x)$ en $W_3(x) \rightarrow$ conflict $\Rightarrow T_1 \rightarrow T_3$
- $R_2(x)$ en $W_1(x) \rightarrow$ conflict $\Rightarrow T_2 \rightarrow T_1$
- $R_2(x)$ en $W_3(x) \rightarrow$ conflict $\Rightarrow T_2 \rightarrow T_3$
- $W_1(x)$ en $R_3(x) \rightarrow$ conflict $\Rightarrow T_1 \rightarrow T_3$
- $W_1(x)$ en $W_2(x) \rightarrow$ conflict $\Rightarrow T_1 \rightarrow T_2$
- $W_1(x)$ en $W_3(x) \rightarrow$ conflict $\Rightarrow T_1 \rightarrow T_3$
- $R_3(x)$ en $W_2(x) \rightarrow$ conflict $\Rightarrow T_3 \rightarrow T_2$
- $R_4(y)$ en $W_1(y) \rightarrow$ conflict $\Rightarrow T_4 \rightarrow T_1$
- $R_5(y)$ en $W_1(y) \rightarrow$ conflict $\Rightarrow T_5 \rightarrow T_1$
- $R_5(y)$ en $W_4(y) \rightarrow$ conflict $\Rightarrow T_5 \rightarrow T_4$

Stap 2: graph (unieke edges)

$$T_1 \rightarrow T_2,\ T_1 \rightarrow T_3,\ T_2 \rightarrow T_1,\ T_2 \rightarrow T_3,\ T_3 \rightarrow T_2,\ T_4 \rightarrow T_1,\ T_5 \rightarrow T_1,\ T_5 \rightarrow T_4$$

Stap 3: conclusie
We zien direct cycles:

$$T_1 \leftrightarrow T_2 \quad \text{en} \quad T_2 \leftrightarrow T_3$$

$\Rightarrow$ **schedule is niet serialisable**

Complete Precedence Graph

Op basis van de gevonden edges construeren we de precedence graph.

Nodes:

$$\{T_1, T_2, T_3, T_4, T_5\}$$

Edges:

$$T_1 \rightarrow T_2,\ T_1 \rightarrow T_3,\ T_2 \rightarrow T_1,\ T_2 \rightarrow T_3,\ T_3 \rightarrow T_2,\ T_4 \rightarrow T_1,\ T_5 \rightarrow T_1,\ T_5 \rightarrow T_4$$

Grafische representatie (schematisch):

$$\begin{array}{ccccccc} T_5 & \rightarrow & T_4 & \rightarrow & T_1 & \leftrightarrow & T_2 \\ & & & & \searrow & & \updownarrow \\ & & & & & & T_3 \end{array}$$

8 | SQL

SQL (Structured Query Language) is een gestandaardiseerde programmeertaal die wordt gebruikt voor het beheren en manipuleren van relationele databases.
DQL means Data Query Language, which is used for querying and retrieving data from a database.
DML means Data Manipulation Language, which is used for inserting, updating, and deleting data in a database.
DDL means Data Definition Language, which is used for defining and managing the structure of a database, including creating, altering, and dropping tables and other database objects.
DCL means Data Control Language, which is used for controlling access to data in a database, including granting and revoking permissions to users and roles.

DDL

SQL (Structured Query Language) is een gestandaardiseerde programmeertaal die wordt gebruikt voor het beheren en manipuleren van relationele databases. SQL biedt een krachtige en flexibele manier om gegevens op te vragen, in te voegen, bij te werken en te verwijderen, evenals om de structuur van databases te definiëren en te beheren.

Om SQL uit te leggen gebruik ik deze tabel als voorbeeld:

CREATE TABLE wordt gebruikt om een nieuwe tabel in de database te maken.

```sql
CREATE TABLE Books ( -- de naam van de tabel
    book_id INT PRIMARY KEY, -- een integer kolom die dient als primaire sleutel
    title VARCHAR(255) NOT NULL, -- string, maximaal 255 tekens, mag niet null zijn
    author VARCHAR(255) NOT NULL,
    year INT,
    genre VARCHAR(100)
);
```

↓

book_id	title	author	year	genre

INSERT INTO wordt gebruikt om nieuwe rijen (records) toe te voegen aan een tabel. Je moet strings tussen aanhalingstekens zetten, en getallen niet.

```sql
INSERT INTO Books (book_id, title, author, year, genre) VALUES -- naam en kolommen waarin je wilt
    invoegen
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', 1925, 'Fiction'),
(2, 'To Kill a Mockingbird', 'Harper Lee', 1960, 'Fiction'),
(3, 'A Brief History of Time', 'Stephen Hawking', 1988, 'Science'),
(4, 'The Art of Computer Programming', 'Donald Knuth', 1968, 'Computer Science'),
(5, 'The Lord of the Rings', 'J.R.R. Tolkien', 1954, 'Fantasy');
```

↓

book_id	title	author	year	genre
1	The Great Gatsby	F. Scott Fitzgerald	1925	Fiction
2	To Kill a Mockingbird	Harper Lee	1960	Fiction
3	A Brief History of Time	Stephen Hawking	1988	Science
4	The Art of Computer Programming	Donald Knuth	1968	Computer Science
5	The Lord of the Rings	J.R.R. Tolkien	1954	Fantasy

UPDATE wordt gebruikt om bestaande records in een tabel bij te werken. Je moet een WHERE-clausule gebruiken om aan te geven welke records je wilt bijwerken, anders worden alle records bijgewerkt.

```
UPDATE Books
SET genre = 'Classic Fiction' -- de nieuwe waarde die je wilt instellen
WHERE book_id = 1; -- de voorwaarde om te bepalen welke record(s) worden bijgewerkt
```

↓

book_id	title	author	year	genre
1	The Great Gatsby	F. Scott Fitzgerald	1925	**Classic Fiction**
2	To Kill a Mockingbird	Harper Lee	1960	Fiction
3	A Brief History of Time	Stephen Hawking	1988	Science
4	The Art of Computer Programming	Donald Knuth	1968	Computer Science
5	The Lord of the Rings	J.R.R. Tolkien	1954	Fantasy

DELETE wordt gebruikt om records uit een tabel te verwijderen. Je moet een WHERE-clausule gebruiken om aan te geven welke records je wilt verwijderen, anders worden alle records verwijderd.

```
DELETE FROM Books
WHERE book_id = 2; -- de voorwaarde om te bepalen welke record(s) worden verwijderd
```

↓

book_id	title	author	year	genre
1	The Great Gatsby	F. Scott Fitzgerald	1925	Fiction
3	A Brief History of Time	Stephen Hawking	1988	Science
4	The Art of Computer Programming	Donald Knuth	1968	Computer Science
5	The Lord of the Rings	J.R.R. Tolkien	1954	Fantasy

SELECT wordt gebruikt om gegevens op te vragen uit een tabel. Je kunt specifieke kolommen selecteren, alle kolommen selecteren, en je kunt ook voorwaarden gebruiken om te filteren welke rijen worden geretourneerd.

```
SELECT title, author
FROM Books
WHERE genre = 'Science'; -- de voorwaarde om te bepalen welke record(s) worden geselecteerd
```

↓

title	author
A Brief History of Time	Stephen Hawking

Andere keywords die je vaak zult tegenkomen in SQL zijn:

- **JOIN** wordt gebruikt om rijen van twee of meer tabellen te combineren op basis van een gerelateerde kolom tussen hen.

- **GROUP BY** wordt gebruikt in combinatie met aggregatiefuncties (zoals COUNT, SUM, AVG, MAX, MIN) om rijen te groeperen die dezelfde waarden hebben in opgegeven kolommen.
- **ORDER BY** wordt gebruikt om de resultaten van een query te sorteren op basis van een of meer kolommen, in oplopende (ASC) of
- **IN** wordt gebruikt in een WHERE-clausule om te controleren of een waarde overeenkomt met een waarde in een lijst van waarden.
- **LIKE** wordt gebruikt in een WHERE-clausule om te zoeken naar een opgegeven patroon in een kolom.
- **DISTINCT** wordt gebruikt in een SELECT-query om dubbele rijen uit het resultaat te verwijderen, zodat alleen unieke
- **EXISTS** wordt gebruikt in een WHERE-clausule om te controleren of een subquery resultaten oplevert.

Ook heeft DDL constraints zoals PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, en DEFAULT die worden gebruikt om regels en beperkingen op te leggen aan de gegevens in de database. Deze kan je op keys zetten, zoals

```
CONSTRAINT Book_pk PRIMARY KEY (book_id)
```

of op andere kolommen, zoals

```
CONSTRAINT Year_check CHECK (year > 0)
```

Views

zijn virtuele tabellen die worden gedefinieerd door een SQL-query. Ze bevatten geen fysieke gegevens, maar tonen in plaats daarvan de resultaten van de query wanneer ze worden opgevraagd.

```
CREATE VIEW ScienceBooks AS
SELECT title, author
FROM Books
WHERE genre = 'Science';
```

Nu kan je de view gebruiken alsof het een echte tabel is:

```
SELECT * FROM ScienceBooks;
```

DML (triggers)

zijn speciale soorten opgeslagen procedures die automatisch worden uitgevoerd (of "getriggerd") wanneer bepaalde gebeurtenissen plaatsvinden in de database, zoals het invoegen, bijwerken of verwijderen van records in een tabel. Triggers worden vaak gebruikt om de integriteit van gegevens te handhaven, automatisch berekeningen uit te voeren, of om auditlogs bij te houden van wijzigingen in de database.

```
CREATE TRIGGER UpdateBookCount
AFTER INSERT ON Books
FOR EACH ROW
BEGIN
    UPDATE BookCounts
    SET count = count + 1
    WHERE genre = NEW.genre;
END;
```

DCL

DCL (Data Control Language) is een subset van SQL die wordt gebruikt om de toegang tot gegevens in een database te beheren en te controleren. De belangrijkste DCL-commando's zijn GRANT en REVOKE, die worden gebruikt om gebruikersrechten toe te kennen of in te trekken voor specifieke database-objecten zoals tabellen, views, of procedures.

SQL Injection

SQL-injectie is een beveiligingsaanval waarbij een aanvaller kwaadaardige SQL-code in een invoerveld van een webapplicatie injecteert, met als doel de database te manipuleren, gegevens te stelen, of zelfs volledige controle over de database te krijgen. Een voorbeeld hiervan is wanneer een webapplicatie een SQL-query uitvoert op basis van gebruikersinvoer zonder deze correct te valideren of te ontsmetten, waardoor een aanvaller bijvoorbeeld een query kan injecteren die alle gebruikersgegevens retourneert of een nieuwe gebruiker met beheerdersrechten kan aanmaken.

```
-- Kwetsbare code
String query = "SELECT * FROM Users WHERE username = '" + userInput + "' AND password = '" +
    passwordInput + "'";
```

In dit voorbeeld kan een aanvaller de ‘userInput‘ of ‘passwordInput‘ manipuleren om kwaadaardige SQL-code in te voeren, zoals ‘’; DROP TABLE Users; –‘, wat zou resulteren in het verwijderen van de hele ‘Users‘-tabel als de query wordt uitgevoerd.

9 | External merge sort en MapReduce

External merge sort

External sorting wordt gebruikt wanneer de dataset te groot is om volledig in het interne geheugen (RAM) te passen. In plaats daarvan wordt gewerkt met blokken data die tussen schijf en geheugen worden verplaatst.

Kernidee

Het algoritme is een variant van merge sort en bestaat uit twee fasen:
- **Fase 1: opdelen en sorteren**
- **Fase 2: samenvoegen (merge)**

Stappenplan

Fase 1: Maak gesorteerde sublijsten

1. Lees een chunk van de data in het interne geheugen (zoveel als past).
2. Sorteer deze data met een intern sorteeralgoritme.
3. Schrijf de gesorteerde data weg als een **sublist** op schijf.
4. Herhaal dit totdat de volledige dataset is verwerkt.

Je hebt nu meerdere gesorteerde sublijsten op schijf, hiermee ga je verder door met de volgende stap.

Fase 2: Merge de sublijsten

1. Laad van elke sublist het eerste blok in het geheugen.
2. Vergelijk de eerste elementen van elke sublist.
3. Schrijf het kleinste element naar de output.
4. Vervang dit element door het volgende element uit dezelfde sublist.
5. Als een blok leeg is, laad een nieuw blok van die sublist.
6. Herhaal dit totdat alle sublijsten leeg zijn.

Tada! één volledig gesorteerde outputfile.

MapReduce

Cheatsheet

1 · RELATIONEEL MODEL

Kernbegrippen

- **Relatieschema** $R(A_1, \ldots, A_n)$: naam + attributen
- **Tuple**: één rij; **degree** = aantal attributen
- **Superkey**: set K zodat geen twee tuples gelijk zijn op K
- **Candidate key**: minimale superkey (A^+ omvat alle attr.)
- **Primary key**: gekozen candidate key; mag niet NULL zijn
- **Foreign key**: K in R_i verwijst naar PK van R_j

Relationele algebra — operators

Symbool	Betekenis & voorbeeld
$\sigma_p(R)$	**Selectie**: behoud alleen rijen waarvoor predicaat p geldt. $\sigma_{\text{year}>2000}(\text{Movie})$ geeft alle films na 2000.
$\pi_L(R)$	**Projectie**: behoud alleen de kolommen in lijst L. $\pi_{\text{title,year}}(\text{Movie})$ geeft alleen titel en jaar. Verwijdert duplicaten.
$R \cup S$	**Union**: alle unieke tuples uit R én S. Schema's moeten gelijk zijn.
$R \cap S$	**Intersection**: alleen tuples die in zowel R als S voorkomen.
$R - S$	**Difference**: tuples die in R zitten maar *niet* in S.
$R \times S$	**Cartesisch product**: elke tuple van R gecombineerd met elke tuple van S. Resultaat heeft $\lvert R\rvert \times \lvert S\rvert$ rijen.
$R \bowtie_\theta S$	**Theta-join**: cartesisch product gefilterd op conditie θ. Voorbeeld: $R \bowtie_{R.id=S.id} S$.
$R \bowtie S$	**Natural join**: join op alle gelijknamige attributen; dubbele kolommen worden samengevoegd.
$R \div S$	**Division**: geeft tuples uit R die gecombineerd met *elke* tuple in S in R voorkomen. Handig voor "voor alle"-vragen.
$\rho(T)(\text{expr})$	**Renaming**: geeft het resultaat tijdelijk een nieuwe naam T.
$\tau_{\uparrow B \downarrow A}(R)$	**Sorteren**: eerst op B oplopend, dan op A aflopend (XRA).
$\Gamma_{A,\, f(B)}(R)$	**Groeperen**: groepeer op A, bereken aggregaat f (MIN, MAX, COUNT, ...) over B per groep (XRA).

Optimalisatietip: pas σ en π zo vroeg mogelijk toe, vóór joins en cartesische producten. Zo is de hoeveelheid data bij de zware operators zo klein mogelijk.

2 · ENTITY-RELATIONSHIP DIAGRAM (ERD)

Een ERD is een grafische weergave van het domein *vóór* je een database bouwt. Het toont welke entiteiten bestaan, welke attributen ze hebben en hoe ze zich tot elkaar verhouden.

Componenten

- **Rechthoek** → Entiteit (een "ding" in het domein, bv. Student, Film).
- **Ovaal** → Attribuut van een entiteit (bv. naam, geboortedatum). Onderstreept ovaal = sleutelattribuut.
- **Dubbel ovaal** → Meerwaardig attribuut (kan meerdere waarden hebben, bv. telefoonnummers).
- **Ruit** → Relatie tussen twee of meer entiteiten (bv. "speelt in", "werkt bij").
- **Dubbele rechthoek** → Zwakke entiteit: kan alleen bestaan in relatie tot een andere entiteit en heeft geen volledige eigen sleutel. De gedeeltelijke sleutel (gestippeld ovaal) vormt samen met de sleutel van de sterke entiteit een unieke id.
- **Dubbele ruit** → Identificerende relatie voor een zwakke entiteit.

Kardinaliteiten geven aan hoeveel entiteiten aan elke kant van een relatie kunnen deelnemen:

- **1:1** — Eén entiteit van type A hoort bij precies één van type B. Bv. persoon–paspoort.
- **1:N** — Één entiteit van type A hoort bij meerdere van type B. Bv. docent–cursus.
- **M:N** — Meerdere van A horen bij meerdere van B. Bv. student–vak. Vereist een koppeltabel in het relationele model.

Deelname:

- $(1, \ldots)$ = verplicht: elke entiteit *moet* deelnemen aan de relatie.
- $(0, \ldots)$ = optioneel: deelname is niet verplicht.

3 · FUNCTIONAL DEPENDENCIES (FD)

$X \to Y$ betekent: als twee tuples dezelfde waarden hebben voor X, dan hebben ze ook dezelfde waarden voor Y. Lees: "X bepaalt Y".

Closure A^+ berekenen — welke attributen worden bepaald door A?

1. Begin met $A^+ = \{A\}$ (een attribuut bepaalt altijd zichzelf).
2. Zoek een FD $X \to Y$ in F waarbij $X \subseteq A^+$. Voeg Y toe aan A^+.
3. Herhaal stap 2 totdat er niets meer bij komt.
4. Is A^+ = alle attributen van R? Dan is A een **candidate key**.

Let op: $(AB)^+$ is de closure van de combinatie AB, *niet* $A(B^+)$.

Verliesvrije decompositie (XYZ-regel)
Splits R in R_1 en R_2. Definieer:

$$X = R_1 \cap R_2, \quad Y = R_1 - X, \quad Z = R_2 - X$$

De decompositie is **verliesvrij** als geldt: $X \to Y$ **of** $X \to Z$ (in F^+).
Intuïtie: de doorsnede X fungeert als een soort sleutel die de join eenduidig maakt.

Dependency Preserving (DP)
Na decompositie in $R_1, R_2, \ldots$ moet je de originele FDs kunnen controleren *binnen één enkele deelrelatie* (zonder join). Formeel: de FDs die je per deelrelatie kunt afleiden moeten samen equivalent zijn aan de originele set F.
Een decompositie kan verliesvrij zijn zonder DP te zijn, en andersom.

4 · NORMALISATIE

Normaliseren = relaties opsplitsen om **redundantie** te verminderen, zonder informatie te verliezen (lossless).

Normaalvormen overzicht

NF	Eis
1NF	Alle attribuutwaarden zijn atomair (geen lijsten of herhaalde groepen).
2NF	Voldoet aan 1NF. Geen enkel niet-prime attribuut mag partieel afhankelijk zijn van een candidate key (d.w.z. afhankelijk van slechts een *deel* van een samengestelde sleutel).
3NF	Voldoet aan 2NF. Geen transitieve afhankelijkheid voor niet-prime attributen: $A \to B \to C$ is verboden als B geen superkey is en C geen prime attribuut. Uitzondering: $X \to Y$ mag als X superkey is *of* als Y een prime attribuut is.
BCNF	Voldoet aan 3NF. Voor *elke* FD $X \to Y$ geldt: X is een superkey. Strenger dan 3NF: geen uitzonderingen voor prime attributen. Snel checken: is de linkerkant van elke FD een candidate key?
4NF	Voldoet aan BCNF. Geen non-triviale multivalued dependency $X \twoheadrightarrow Y$ tenzij X een superkey is. MVD: voor één waarde van X zijn meerdere waarden van Y mogelijk, onafhankelijk van de rest. Oplossing: splits in $R_1(X, Y)$ en $R_2(X, Z)$.

Als 3NF niet geldt, geldt BCNF ook niet. BCNF garandeert geen DP; 3NF wel (altijd te behalen met DP).

Minimal Cover berekenen (= minimale equivalente FD-set)

1. **RHS splitsen**: schrijf elke FD $X \to AB$ om naar $X \to A$ en $X \to B$.
2. **LHS minimaliseren**: voor elke FD $XY \to A$, kijk of $X \to A$ al volgt (via closure van X zonder Y). Zo ja: verwijder Y uit de linkerkant.
3. **Overbodige FDs verwijderen**: voor elke FD, kijk of ze afleidbaar is uit de overige FDs. Zo ja: verwijder haar.

Volgorde stap 3 doet ertoe: verwijder FDs één voor één en herbereken telkens.

5 · RECOVERY & LOGGING

Bij een crash moet de database worden hersteld naar een consistente toestand. Logging legt bij wat er is veranderd, zodat je dit kunt ongedaan maken (UNDO) of opnieuw toepassen (REDO).

Type	Volgorde op schijf	UNDO?	REDO?
UNDO	log → data → COMMIT	Ja	Nee
REDO	log → COMMIT → data	Nee	Ja
UNDO/REDO	log → data (COMMIT overal)	Ja	Ja

Logrecord-formaat: `<T, X, oude_waarde, nieuwe_waarde>`
Bij UNDO: herstel naar *oude* waarde. Bij REDO: schrijf *nieuwe* waarde.

Non-quiescent checkpoint aanmaken (UNDO/REDO):

1. Schrijf `<START CKPT(T1,...,Tk)>` met alle op dat moment actieve transacties, en flush de log naar schijf.
2. Flush alle dirty buffers die al *vóór* START CKPT waren aangemaakt naar schijf (buffers van ná START CKPT hoef je nog niet te flushen).
3. Schrijf `<END CKPT>` en flush de log opnieuw.

Herstelalgoritme na crash:

1. Scan de log **achteruit** vanuit het laatste record.
2. **END CKPT gevonden?** Scan verder terug tot het bijbehorende `START CKPT`. Transacties gestart vóór START CKPT die al gecommit waren, hoeven *niet* geREDO'd te worden (al geflusht).
3. **Geen END CKPT gevonden?** (Crash tijdens checkpointing.) Scan terug tot het absolute begin van de log.
4. **REDO** alle committed transacties, vroegste eerst (voorwaartse scan).
5. **UNDO** alle uncommitted transacties, laatste eerst (achterwaartse scan).

6 · 2-PHASE COMMIT PROTOCOL (2PC)

2PC coördineert een gedistribueerde transactie over meerdere nodes, zodat alle nodes dezelfde beslissing nemen (commit of abort). Er is één **coördinator** (COORD) en meerdere **participanten** (PARTs).

Fase 1 — Stemmen

1. COORD schrijft `<prepare T>` in zijn log, stuurt `[prepare T]` naar alle PARTs.
2. Elke PART beslist: kan hij committen? Zo ja: log `<Ready T>`, stuur `[ready T]`. Zo nee: log abort, stuur `[abort T]` en rollback lokaal.

Fase 2 — Beslissing & uitvoering

1. Alle stemmen positief → COORD logt `<Commit T>`, stuurt `[Commit T]` naar iedereen.
2. Eén stem negatief → COORD logt `<Abort T>`, stuurt `[Abort T]` naar alle PARTs die `[ready]` stemden.
3. Elke PART die `[ready T]` stemde, wacht op de beslissing, logt die en handelt ernaar.

Herstel bij timeout:

Log-status	Rol	Actie
Commit/Abort aanwezig	beide	Beslissing al genomen — gewoon uitvoeren.
`<Prepare T>`, geen beslissing	COORD	Nog niet beslist — forceer Abort en herstart protocol.
Geen `<Ready T>`	PART	Nog niet gestemd — forceer Abort.
`<Ready T>`, geen beslissing	PART	Vraag COORD om beslissing. Geen reactie → start **CTP**.

Cooperative Termination Protocol (CTP):
Kies een leider onder de overgebleven PARTs. De leider vraagt de status bij alle andere PARTs op:

- Iemand kent de beslissing → broadcast die beslissing aan iedereen.
- Iemand heeft *nog niet* gestemd → broadcast Abort (hij kon immers nog terug).
- Iedereen heeft positief gestemd, maar niemand kent de beslissing → **GEBLOKKEERD**: menselijke interventie nodig.

2PC is een blocking protocol. 3PC vermindert de kans op blokkering maar kan het niet volledig elimineren bij netwerkstoringen.

7 · CONCURRENCY

Concurrency = meerdere transacties draaien tegelijkertijd (interleaved). Doel: interleaved schedules moeten hetzelfde resultaat geven als een *seriële* uitvoering.

Soorten schedules:

- **Serial schedule**: transacties volledig na elkaar — altijd correct, maar niet efficiënt.
- **Interleaved schedule**: operaties door elkaar — efficiënter, maar kan conflicten geven.
- **Serialisable schedule**: een interleaved schedule die equivalent is aan *een of ander* serieel schedule — dit is wat we willen.

Conflict definitie: twee operaties conflicteren als ze:

1. Op hetzelfde data-item werken, *én*
2. Van *verschillende* transacties zijn, *én*
3. Minstens één van de twee een **write** is.

$R(x)$ & $R(x)$ geen conflict
$R(x)$ & $W(x)$ **conflict**
$W(x)$ & $W(x)$ **conflict**

Conflict-serialisability testen via precedence graph:

1. Maak een chronologische lijst van alle operaties in de schedule.
2. Doorloop elke operatie en vergelijk met alle *eerdere* operaties: voldoen ze aan de drie conflictcriteria? Zo ja: teken een pijl van de oudere naar de nieuwere transactie.
3. Zet alle unieke pijlen in een graaf.
4. **Cyclus aanwezig** → schedule is **niet conflict-serialisable**. **Geen cyclus** → schedule is conflict-serialisable. De topologische volgorde van de graaf geeft een equivalent serieel schedule.

8 · EXTERNAL MERGE SORT

Gebruikt wanneer de dataset te groot is om volledig in RAM te passen. Het algoritme verplaatst data in blokken tussen schijf en geheugen.

Fase 1 — Gesorteerde sublijsten maken

1. Lees een chunk van de data in RAM (zo groot als past).
2. Sorteer deze chunk met een intern sorteeralgoritme.
3. Schrijf de gesorteerde chunk als **sublist** weg naar schijf.
4. Herhaal totdat de volledige dataset is verwerkt. Je hebt nu k gesorteerde sublijsten op schijf.

Fase 2 — Samenvoegen (merge)

1. Laad het eerste blok van elke sublist in het geheugen.
2. Vergelijk de huidige kop-elementen van alle sublijsten.
3. Schrijf het kleinste element naar de outputfile.
4. Vervang dat element door het volgende element uit dezelfde sublist.
5. Als een blok leeg is: laad het volgende blok van die sublist van schijf.
6. Herhaal stappen 2–5 totdat alle sublijsten volledig zijn verwerkt.

Resultaat: één volledig gesorteerde outputfile.

Met B bufferblokken kun je in één merge-ronde $B-1$ sublijsten samenvoegen (1 blok voor output). Bij heel veel sublijsten zijn meerdere merge-rondes nodig.

9 · SNELLE FORMULES & TRUCJES

Onderwerp	Truc / ezelsbruggetje
Closure A^+	Begin {A}, pas FDs toe tot stabiel.
Candidate key	A^+ = alle attributen van R.
BCNF check	Elke LHS van een FD is superkey? Dan BCNF.
3NF vs BCNF	3NF staat prime-attribuut rechts toe; BCNF niet. 3NF is makkelijker te halen met DP.
Verliesvrij	$X \to Y$ of $X \to Z$ (X = doorsnede $R_1 \cap R_2$)
DP	Alle originele FDs vallen binnen één deelrelatie.
Min. cover	(1) RHS splitsen (2) LHS minimeren (3) redundante FDs weg.
Serialisable	Geen cyclus in precedence graph.
REDO	Committed transacties herstellen (data nog niet weggeschreven).
UNDO	Uncommitted transacties terugdraaien.
2PC blocked	Alle PARTs `ready`, COORD dood → CTP kan niet besluiten → geblokkeerd.
Ext. sort	Fase 1: maak k sublijsten. Fase 2: merge met $B-1$ buffers per ronde.